SQL COMMANDS

Complete Study Notes & Solved Query Bank

OAVS Computer Teacher (TGT) – Exam Preparation

Topics Covered (strictly as per syllabus):

1. Create / Drop Database
2. Create / Modify / Alter / Drop Table
3. DML Commands – INSERT, UPDATE, DELETE, SELECT
4. Aggregate Functions – MAX, MIN, AVG, COUNT, SUM
5. GROUP BY
6. ORDER BY
7. HAVING

📖 Introduction to SQL Commands

SQL (Structured Query Language) is the standard language used to create, manage and work with data stored in a Relational Database Management System (RDBMS) such as MySQL, Oracle, or SQL Server. Every action a database needs – building a database, shaping a table, or working with the rows inside it – is carried out using a small, well-defined set of SQL commands.

SQL commands are broadly divided into two families covered in this chapter:

Category	Full Form	What it Controls	Commands in this Chapter
DDL	Data Definition Language	The structure (skeleton) of the database/table	CREATE, ALTER/MODIFY, DROP
DML	Data Manipulation Language	The actual data (rows) stored inside a table	INSERT, UPDATE, DELETE, SELECT

Once data exists inside a table, SQL also provides powerful tools to **summarise** it using **Aggregate Functions** (COUNT, SUM, AVG, MAX, MIN), to **group** it meaningfully using **GROUP BY**, to **sort** the final result using **ORDER BY**, and to **filter those groups** using **HAVING**. Together, these eleven commands/clauses form a complete, exam-ready foundation of SQL – from creating a database to writing summarised, sorted, filtered queries on it.

Exam Point: A quick way to remember the split: DDL commands define WHERE data lives (structure); DML commands work WITH the data itself (rows and values).

Remember: This chapter builds everything step by step – Database → Table → Data (INSERT/UPDATE/DELETE/SELECT) → Summarise (Aggregate + GROUP BY + HAVING) → Sort (ORDER BY). Learning them in this order makes each new topic easier to follow.

💾 Sample Database Used Throughout This Book

To keep every example consistent and exam-friendly, **one single table –** `Employee` – is used across INSERT, UPDATE, DELETE, SELECT, Aggregate Functions, GROUP BY, ORDER BY and HAVING. Wherever a query changes this data (INSERT/UPDATE/DELETE), the change is noted – but all SELECT-family examples assume the original data shown below unless stated otherwise.

Table: Employee

EmpID	Name	Dept	Salary	City	JoinDate
1	Anil	IT	45000	Bhubaneswar	2019-05-10
2	Sunita	HR	38000	Cuttack	2020-03-15
3	Ramesh	IT	52000	Rourkela	2018-07-22
4	Priya	Sales	30000	Bhubaneswar	2021-01-05
5	Kiran	Finance	60000	Puri	2017-11-30
6	Manoj	Sales	33000	Cuttack	2020-09-18
7	Deepa	HR	41000	Rourkela	2019-02-25
8	Suresh	IT	48000	Bhubaneswar	2022-06-01
9	Pooja	Finance	55000	Puri	2018-04-14
10	Ajay	Sales	29000	Rourkela	2021-08-09

Remember: Dept has 3 values: IT (3 rows), HR (2 rows), Sales (3 rows), Finance (2 rows). This grouping pattern is used in every GROUP BY / HAVING example ahead.

1 CREATE DATABASE / DROP DATABASE

1.1 CREATE DATABASE

Purpose: Creates a new, empty database on the SQL server. It is the very first DDL (Data Definition Language) step before any table can be made.

Syntax

```
CREATE DATABASE database_name;
```

Explanation of Syntax

- `CREATE DATABASE` – keyword pair that tells the server to allocate a new database.
- `database_name` – the name you choose (no spaces, cannot start with a digit).

Example

Question: Create a new database named SchoolDB.

```
CREATE DATABASE SchoolDB;
```

Expected Result: A new empty database named `SchoolDB` is created. Output message: "*Query OK, 1 row affected*" (varies by RDBMS).

Exam Point: CREATE DATABASE is a DDL command – it is auto-committed and cannot be rolled back in most RDBMS.

Common Mistake: Trying to CREATE DATABASE with a name that already exists causes an error. Use IF NOT EXISTS to avoid this: CREATE DATABASE IF NOT EXISTS SchoolDB;

1.2 DROP DATABASE

Purpose: Permanently deletes an entire database along with all its tables, data, indexes and views.

Syntax

```
DROP DATABASE database_name;
```

Explanation of Syntax

- `DROP DATABASE` – keyword pair that removes the database and frees the disk space.
- `database_name` – the database to be deleted.

Example

Question: Permanently delete the database SchoolDB.

```
DROP DATABASE SchoolDB;
```

Expected Result: SchoolDB and everything inside it is permanently removed.

Remember: DROP removes the structure AND the data permanently. There is no UNDO.

Common Mistake: Confusing DROP DATABASE (deletes the whole database) with DROP TABLE (deletes only one table). Exam questions often test this difference.

Shortcut: Memory trick – CREATE = birth of database, DROP = death of database (nothing survives).

2 CREATE TABLE / ALTER (MODIFY) TABLE / DROP TABLE

2.1 CREATE TABLE

Purpose: Defines a new table – its name, columns and datatypes – inside the current database.

Syntax

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ...
);
```

Explanation of Syntax

- `table_name` – unique name for the table inside the database.
- `column datatype` pairs – each column must have a name and a datatype (INT, VARCHAR(n), DATE, DECIMAL, etc.).
- Columns are separated by commas; the whole list is inside round brackets.

Example – Creating the Employee table used in this book

Question: Create an Employee table with columns EmpID, Name, Dept, Salary, City and JoinDate.

```
CREATE TABLE Employee (
    EmpID     INT PRIMARY KEY,
    Name      VARCHAR(30),
    Dept      VARCHAR(20),
    Salary    DECIMAL(10,2),
    City      VARCHAR(20),
    JoinDate  DATE
);
```

Expected Result: An empty table named Employee is created with 6 columns and the given datatypes; no rows exist yet until INSERT is used.

Exam Point: CREATE TABLE only defines structure. It does NOT insert any data – that is the job of INSERT (a DML command).

Common Mistake: Forgetting a datatype for a column, or using a duplicate column name, causes a syntax error.

2.2 ALTER TABLE (with MODIFY / ADD / DROP COLUMN)

Purpose: Changes the structure of an existing table – add a new column, change (modify) a column's datatype, or remove a column – without losing existing data.

Syntax

```
-- Add a new column
ALTER TABLE table_name ADD column_name datatype;

-- Modify / change datatype of an existing column
ALTER TABLE table_name MODIFY column_name new_datatype;

-- Drop (remove) a column
ALTER TABLE table_name DROP COLUMN column_name;
```

Explanation of Syntax

- `ADD` – appends a brand-new column (existing rows get NULL in that column by default).
- `MODIFY` – changes the datatype/size of a column that already exists (in SQL Server the keyword is `ALTER COLUMN` instead of MODIFY).
- `DROP COLUMN` – permanently removes a column and its data from every row.

e.g. – MODIFY vs ALTER COLUMN (same job, different RDBMS keyword)

```
-- MySQL
ALTER TABLE Employee MODIFY Salary DECIMAL(12,2);

-- SQL Server
ALTER TABLE Employee ALTER COLUMN Salary DECIMAL(12,2);
```

Both statements do exactly the same thing – they change the Salary column's datatype to DECIMAL(12,2). Only the keyword (MODIFY vs ALTER COLUMN) changes depending on the RDBMS; the rest of the ALTER TABLE statement stays identical.

Example 1 – Add a column

Question: Add a new column Email to the Employee table.

```
ALTER TABLE Employee ADD Email VARCHAR(40);
```

Expected Result: Employee table now has 7 columns; the new Email column contains NULL for all existing rows.

Example 2 – Modify a column's datatype

Question: Change the Salary column so that it can store larger values.

```
ALTER TABLE Employee MODIFY Salary DECIMAL(12,2);
```

Expected Result: The Salary column can now store larger values; existing salary data is preserved.

Example 3 – Drop a column

Question: Remove the Email column from the Employee table.

```
ALTER TABLE Employee DROP COLUMN Email;
```

Expected Result: The Email column and all its values are permanently removed from the table.

Exam Point: ALTER is DDL. ADD, MODIFY and DROP COLUMN are all clauses used WITH the ALTER TABLE statement – they are not separate commands.

Common Mistake: Writing ALTER COLUMN in MySQL (correct keyword is MODIFY) or writing MODIFY in SQL Server (correct keyword is ALTER COLUMN) – syntax differs slightly by RDBMS, but the concept tested in exams is the same.

2.3 DROP TABLE

Purpose: Permanently deletes an entire table – its structure AND all its data – from the database.

Syntax

```
DROP TABLE table_name;
```

Example

Question: Delete the entire Employee table (structure and data).

```
DROP TABLE Employee;
```

Expected Result: The Employee table no longer exists; every row of data is gone permanently.

Remember: CREATE builds structure, ALTER/MODIFY changes structure, DROP destroys structure completely. All three are DDL commands and auto-commit.

Shortcut: C-A-D rule: Create → Alter (Modify) → Drop = the full life-cycle of a table.

3 DML Commands – Overview

DML (Data Manipulation Language) commands work with the *data* stored inside tables, not with the table structure. The four DML commands covered in this book are:

Command	Purpose	Changes Data?
INSERT	Add new row(s) into a table	Yes
UPDATE	Modify existing row(s)	Yes
DELETE	Remove existing row(s)	Yes
SELECT	Retrieve / view data (does not change anything)	No

> **Exam Point:** DDL (CREATE, ALTER, DROP) changes structure and auto-commits. DML (INSERT, UPDATE, DELETE, SELECT) changes/reads data and can normally be rolled back before COMMIT (transactions are outside this book's scope, but the concept is a common MCQ).

3.1 INSERT Command

Purpose: Adds one or more new rows of data into a table.

Syntax

```
-- Insert into ALL columns (values must follow the table's column order)
INSERT INTO table_name VALUES (value1, value2, ...);

-- Insert into SPECIFIC columns
INSERT INTO table_name (col1, col2, ...) VALUES (value1, value2, ...);

-- Insert MULTIPLE rows in one statement
INSERT INTO table_name VALUES (row1...), (row2...), (row3...);
```

Explanation of Syntax

- `INTO table_name` – names the table receiving the data.
- `(col1, col2, ...)` – optional column list; if omitted, values must be given for EVERY column in the exact order the table was created.
- `VALUES (...)` – the actual data; text values need single quotes, numbers do not.

Example 1 – Insert a full row

Question: Insert a new employee: EmpID 1, Anil, IT, 45000, Bhubaneswar, 2019-05-10.

```
INSERT INTO Employee VALUES
(1, 'Anil', 'IT', 45000, 'Bhubaneswar', '2019-05-10');
```

Expected Result: One new row (EmpID 1, Anil) is added to Employee.

Example 2 – Insert into specific columns only

Question: Insert an employee giving only EmpID 11, Name Rina and Dept HR.

```
INSERT INTO Employee (EmpID, Name, Dept)
VALUES (11, 'Rina', 'HR');
```

Expected Result: A new row is added where Salary, City and JoinDate are automatically set to NULL.

Example 3 – Insert multiple rows at once

Question: Insert two employees (Sunita and Ramesh) using a single INSERT statement.

```
INSERT INTO Employee VALUES
(2, 'Sunita', 'HR', 38000, 'Cuttack', '2020-03-15'),
(3, 'Ramesh', 'IT', 52000, 'Rourkela', '2018-07-22');
```

Expected Result: Two rows are inserted in a single statement, saving repeated INSERT INTO ... calls.

Exam Point: INSERT is a DML command. Text/date values always need single quotes; numeric values never need quotes.

Common Mistake: Mismatched column count – the number of values must exactly equal the number of columns listed (or all columns if none listed).

Shortcut: If you name the columns, order does not matter. If you skip the column list, order must exactly match the CREATE TABLE order.

3.2 UPDATE Command

Purpose: Modifies the values of existing row(s) in a table.

Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Explanation of Syntax

- `UPDATE table_name` – names the table to change.
- `SET` – lists the column(s) and their new value(s), comma-separated for multiple columns.
- `WHERE condition` – selects exactly which row(s) get updated. **If omitted, ALL rows are updated.**

Example 1 – Update one column of one row

Question: Change Anil's (EmpID 1) salary to 50000.

```
UPDATE Employee
SET Salary = 50000
WHERE EmpID = 1;
```

Expected Result: Only Anil's (EmpID 1) Salary changes from 45000 to 50000; every other row is untouched.

Example 2 – Update multiple columns together

Question: For the employee with EmpID 4, change City to Puri and Dept to Finance.

```
UPDATE Employee
SET City = 'Puri', Dept = 'Finance'
WHERE EmpID = 4;
```

Expected Result: Priya's City becomes Puri and Dept becomes Finance in the same statement.

Example 3 – Update using an expression (percentage rise)

Question: Give every IT employee a 10% salary raise.

```
UPDATE Employee
SET Salary = Salary + (Salary * 0.10)
WHERE Dept = 'IT';
```

Expected Result: Every IT employee's salary increases by 10% (Anil: 49500, Ramesh: 57200, Suresh: 52800).

Common Mistake: Running UPDATE without a WHERE clause changes EVERY row in the table – one of the most dangerous and most-asked exam traps.

Exam Point: UPDATE is DML. The SET clause can use the column's own old value in an expression, e.g. Salary = Salary + 1000.

3.3 DELETE Command

Purpose: Removes existing row(s) of data from a table (the table structure remains).

Syntax

```
DELETE FROM table_name
WHERE condition;
```

Explanation of Syntax

- `DELETE FROM table_name` – names the table to remove rows from.
- `WHERE condition` – picks the exact row(s) to delete. **If omitted, ALL rows are deleted** (table structure still remains, unlike DROP TABLE).

Example 1 – Delete one specific row

Question: Delete the employee whose EmpID is 10.

```
DELETE FROM Employee
WHERE EmpID = 10;
```

Expected Result: Ajay's row (EmpID 10) is permanently removed; 9 rows remain.

Example 2 – Delete rows matching a condition (possibly multiple)

Question: Delete the Sales employees whose salary is below 30000.

```
DELETE FROM Employee
WHERE Dept = 'Sales' AND Salary < 30000;
```

Expected Result: Rows where Dept is Sales AND Salary is below 30000 are removed (Ajay, Salary 29000).

Common Mistake: DELETE FROM Employee; (no WHERE) empties the entire table – every row is lost, but the table itself still exists (different from DROP TABLE).

Exam Point: DELETE removes data; DROP removes the whole table structure; both are different from UPDATE which only changes values, not removes rows.

3.4 SELECT Command

Purpose: Retrieves (reads) data from a table. It is the most frequently used SQL command and does not change any data.

Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;

-- Select all columns
SELECT * FROM table_name;

-- Select only unique values
SELECT DISTINCT column_name FROM table_name;
```

Explanation of Syntax

- `SELECT column1, column2` – the columns you want to see; use `*` for all columns.
- `FROM table_name` – the table to read from.
- `WHERE condition` – optional filter; only matching rows are returned.
- `DISTINCT` – removes duplicate values from the result.

Example 1 – Select all columns

Question: Display all details of all employees.

```
SELECT * FROM Employee;
```

Expected Result: All 10 rows and all 6 columns of Employee are displayed exactly as stored.

Example 2 – Select specific columns with a WHERE condition

Question: Display the Name and Salary of the IT employees.

```
SELECT Name, Salary
FROM Employee
WHERE Dept = 'IT';
```

Name	Salary
Anil	45000
Ramesh	52000
Suresh	48000

Example 3 – Comparison, logical, BETWEEN, IN and LIKE operators

```
=, >, <, >=, <=, <>      -- comparison operators
AND, OR, NOT             -- logical operators
BETWEEN low AND high     -- range check
IN (v1, v2, ...)         -- match any value in a list
LIKE 'pattern'           -- pattern match ( % = any chars, _ = one char )
```

Question: Display the employees whose salary is between 30000 and 50000.

```
SELECT Name, Salary FROM Employee
WHERE Salary BETWEEN 30000 AND 50000;
```

Name	Salary
Anil	45000
Sunita	38000
Manoj	33000
Deepa	41000
Suresh	48000

Example 3a – IN (matches any value in a list)

Question: Display the employees who work in HR or Finance.

```
SELECT Name, Dept FROM Employee
WHERE Dept IN ('HR', 'Finance');
```

Name	Dept
Sunita	HR
Kiran	Finance
Deepa	HR
Pooja	Finance

Explanation: IN checks if Dept matches ANY value in the list ('HR' or 'Finance') – it is a short form of writing WHERE Dept='HR' OR Dept='Finance'.

Example 3b – LIKE (pattern matching)

Question: Display the names that start with the letter S.

```
SELECT Name FROM Employee
WHERE Name LIKE 'S%';
```

Name
Sunita
Suresh

Explanation: % means "any number of characters after S", so LIKE 'S%' finds every name starting with S.

Question: Display the names whose second letter is a.

```
SELECT Name FROM Employee
WHERE Name LIKE '_a%';
```

Name
Ramesh
Manoj

Explanation: The single underscore _ matches exactly one (any) character, so '_a%' means "2nd letter is a, followed by anything" – Ramesh (R-a-mesh) and Manoj (M-a-noj) both qualify.

Example 4 – DISTINCT

Question: Display the list of different departments (without repeats).

```
SELECT DISTINCT Dept FROM Employee;
```

Dept
IT
HR
Sales
Finance

Exam Point: SELECT is DML but read-only – it never modifies stored data. Execution order (conceptually): FROM → WHERE → SELECT.

Common Mistake: Using = with multiple values instead of IN, e.g. writing WHERE Dept = 'IT' OR Dept = 'HR' can be shortened correctly to WHERE Dept IN ('IT','HR').

Wordy (repetitive, more to type and more room for error):

```
WHERE Dept = 'IT' OR Dept = 'HR' OR Dept = 'Sales' OR Dept = 'Finance';
```

Clean (same result, written using IN):

```
WHERE Dept IN ('IT', 'HR', 'Sales', 'Finance');
```

Both lines return exactly the same rows – IN is simply a shorter, cleaner way to check a column against several possible values, especially useful as the list grows longer.

Shortcut: % in LIKE means "any number of characters"; _ means "exactly one character". e.g. LIKE 'A%' finds names starting with A.

4 Aggregate Functions

Purpose: Aggregate functions perform a calculation on a *set* of values (an entire column, or a group of rows) and return a **single summarised value**. They are most often used with SELECT and GROUP BY.

Function	What it does
COUNT()	Counts the number of rows / non-NULL values
SUM()	Adds up all numeric values in a column
AVG()	Calculates the average (mean) of numeric values
MAX()	Finds the largest value in a column
MIN()	Finds the smallest value in a column

General Syntax

```
SELECT AGG_FUNCTION(column_name) FROM table_name WHERE condition;
```

4.1 COUNT()

Counts rows. `COUNT(*)` counts ALL rows (including NULLs); `COUNT(column)` counts only non-NULL values in that column.

Question: Find the total number of employees.

```
SELECT COUNT(*) FROM Employee;
```

COUNT(*)
10

4.2 SUM()

Adds all values of a numeric column.

Question: Find the total salary of all employees.

```
SELECT SUM(Salary) FROM Employee;
```

SUM(Salary)
431000

4.3 AVG()

Returns the mean (SUM ÷ COUNT) of a numeric column.

Question: Find the average salary of all employees.

```
SELECT AVG(Salary) FROM Employee;
```

AVG(Salary)
43100

4.4 MAX()

Returns the highest value in a column.

Question: Find the highest salary.

```
SELECT MAX(Salary) FROM Employee;
```

MAX(Salary)
60000

4.5 MIN()

Returns the lowest value in a column.

Question: Find the lowest salary.

```
SELECT MIN(Salary) FROM Employee;
```

MIN(Salary)
29000

Exam Point: Aggregate functions ignore NULL values (except COUNT(*), which counts every row regardless of NULLs).

Common Mistake: Mixing a plain column with an aggregate function in SELECT without GROUP BY, e.g. SELECT Name, SUM(Salary) FROM Employee; is invalid in strict SQL mode.

Shortcut: Remember by first letter: C-S-A-M-M – Count, Sum, Avg, Max, Min.

5 GROUP BY Clause

Purpose: Groups rows that share the same value in a column into summary rows, so an aggregate function can be applied to EACH group separately (instead of the whole table at once).

Syntax

```
SELECT column_name, AGG_FUNCTION(column2)
FROM table_name
GROUP BY column_name;
```

Explanation of Syntax

- Every non-aggregated column in SELECT must also appear in GROUP BY.
- GROUP BY runs AFTER WHERE but BEFORE ORDER BY / HAVING in logical query processing.

Example – Number of employees and total salary per department

Question: Find the number of employees and the total salary of each department.

```
SELECT Dept, COUNT(*), SUM(Salary)
FROM Employee
GROUP BY Dept;
```

Dept	COUNT(*)	SUM(Salary)
IT	3	145000
HR	2	79000
Sales	3	92000
Finance	2	115000

Explanation: All rows with the same Dept are collapsed into one summary row; COUNT and SUM are calculated separately for each department.

Exam Point: GROUP BY is almost always used together with an aggregate function; using it alone (without any aggregate) just behaves like DISTINCT.

Common Mistake: Including a column in SELECT that is neither aggregated nor listed in GROUP BY – this causes an error/undefined result.

Wrong (Name is neither aggregated nor in GROUP BY):

```
SELECT Dept, Name, COUNT(*)
FROM Employee
GROUP BY Dept;
```

This gives an error (or an unreliable result) because each Dept group has several names, and SQL cannot decide which single Name to show for the group.

Correct (every selected column is either in GROUP BY or inside an aggregate function):

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept;
```

Dept	COUNT(*)
IT	3
HR	2
Sales	3
Finance	2

6 ORDER BY Clause

Purpose: Sorts the final result set in ascending or descending order of one or more columns.

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

Explanation of Syntax

- `ASC` – ascending order (default, low to high / A to Z).
- `DESC` – descending order (high to low / Z to A).
- Multiple columns: sorting happens by the first column first; the second column only breaks ties.

Example 1 – Ascending (default)

Question: Display Name and Salary, with the lowest salary first.

```
SELECT Name, Salary FROM Employee
ORDER BY Salary;
```

Name	Salary
Ajay	29000
Priya	30000
Manoj	33000
Sunita	38000
Deepa	41000
Anil	45000
Suresh	48000
Ramesh	52000
Pooja	55000
Kiran	60000

Example 2 – Descending

Question: Display Name and Salary, with the highest salary first.

```
SELECT Name, Salary FROM Employee
ORDER BY Salary DESC;
```

Expected Result: Same rows as above, but shown from highest (Kiran, 60000) to lowest (Ajay, 29000).

Example 3 – Sorting by multiple columns

Question: Display employees sorted by Dept (A to Z) and, within each Dept, by Salary (highest first).

```
SELECT Dept, Name, Salary FROM Employee
ORDER BY Dept ASC, Salary DESC;
```

Dept	Name	Salary
Finance	Kiran	60000
Finance	Pooja	55000
HR	Deepa	41000
HR	Sunita	38000
IT	Ramesh	52000
IT	Suresh	48000
IT	Anil	45000
Sales	Manoj	33000
Sales	Priya	30000
Sales	Ajay	29000

Explanation: Rows are grouped alphabetically by Dept first; within each Dept, salaries appear highest to lowest.

Exam Point: ORDER BY is always the LAST clause to execute logically, so it sorts the final result after WHERE, GROUP BY and HAVING have been applied.

Shortcut: ASC = A to Z / small to big (default, no need to type it). DESC must always be typed explicitly.

7 HAVING Clause

Purpose: Filters GROUPS (created by GROUP BY) based on a condition involving an aggregate function – something WHERE cannot do.

Syntax

```
SELECT column_name, AGG_FUNCTION(column2)
FROM table_name
GROUP BY column_name
HAVING AGG_FUNCTION(column2) condition;
```

Explanation of Syntax

- HAVING always comes AFTER GROUP BY (and before ORDER BY, if used).
- The condition in HAVING usually involves an aggregate function like COUNT(), SUM(), AVG().

Example – Departments with more than 2 employees

Question: Find the departments that have more than 2 employees.

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 2;
```

Dept	COUNT(*)
IT	3
Sales	3

Explanation: All 4 departments are first grouped and counted; HAVING then keeps only the groups whose count is greater than 2.

Exam Point: HAVING filters groups AFTER aggregation; WHERE filters individual rows BEFORE aggregation. HAVING cannot be used without GROUP BY in most exam-style questions (though some RDBMS allow it on the whole table as one group).

Common Mistake: Writing WHERE COUNT(*) > 2 instead of HAVING COUNT(*) > 2 – WHERE can NEVER contain an aggregate function directly.

8 WHERE vs HAVING

Point	WHERE	HAVING
Used to filter	Individual rows	Groups (after GROUP BY)
Runs	Before grouping/aggregation	After grouping/aggregation
Aggregate functions allowed?	No (cannot use COUNT, SUM, etc. directly)	Yes (its main purpose)
Can be used without GROUP BY?	Yes, always	Usually used only with GROUP BY
Position in query	After FROM, before GROUP BY	After GROUP BY, before ORDER BY

Remember: Simple line to remember: "WHERE works on ROWS before grouping; HAVING works on GROUPS after grouping."

9 Combined Query Patterns

9.1 Aggregate + GROUP BY

Question: Find the average salary of each city.

```
SELECT City, AVG(Salary)
FROM Employee
GROUP BY City;
```

City	AVG(Salary)
Bhubaneswar	41000.0
Cuttack	35500.0
Rourkela	40666.67
Puri	57500.0

9.2 GROUP BY + HAVING

Question: Find the departments whose average salary is more than 40000.

```
SELECT Dept, AVG(Salary)
FROM Employee
GROUP BY Dept
HAVING AVG(Salary) > 40000;
```

Dept	AVG(Salary)
IT	48333.33
Finance	57500.0

9.3 SELECT + ORDER BY

Question: Display the IT employees from the highest to the lowest salary.

```
SELECT Name, Dept, Salary
FROM Employee
WHERE Dept = 'IT'
ORDER BY Salary DESC;
```

Name	Dept	Salary
Ramesh	IT	52000
Suresh	IT	48000
Anil	IT	45000

9.4 Aggregate + GROUP BY + HAVING + ORDER BY (full combination)

Question: For departments with at least 2 employees, show the employee count and total salary, highest total first.

```
SELECT Dept, COUNT(*), SUM(Salary)
FROM Employee
GROUP BY Dept
HAVING COUNT(*) >= 2
ORDER BY SUM(Salary) DESC;
```

Dept	COUNT(*)	SUM(Salary)
IT	3	145000
Finance	2	115000
Sales	3	92000
HR	2	79000

Exam Point: Logical order of execution: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. This exact sequence is a favourite exam MCQ.

10 Solved Query Bank (32 Questions)

All queries below use the same `Employee` table shown at the start of this book, unless a question is explicitly about creating/altering the table itself.

Solved Query 1

Question: Write a query to create the Employee table.

Table used: Employee (new table)

SQL Query:

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(30),
    Dept VARCHAR(20),
    Salary DECIMAL(10,2),
    City VARCHAR(20),
    JoinDate DATE
);
```

Expected Output:

Table created with 0 rows; structure ready.

Explanation: CREATE TABLE defines the structure; no data exists yet.

Solved Query 2

Question: Add a new column Email to the Employee table.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
ALTER TABLE Employee ADD Email VARCHAR(40);
```

Expected Output:

Employee now has 7 columns; Email = NULL for all existing rows.

Explanation: ALTER TABLE ... ADD inserts a new column without disturbing existing data.

Solved Query 3

Question: Change the Salary column to allow larger decimal values.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
ALTER TABLE Employee MODIFY Salary DECIMAL(12,2);
```

Expected Output:

Salary column datatype updated; existing values unchanged.

Explanation: MODIFY changes a column's datatype/size while keeping stored data intact.

Solved Query 4

Question: Drop the Email column added earlier.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
ALTER TABLE Employee DROP COLUMN Email;
```

Expected Output:

Employee returns to its original 6 columns; Email data is lost.

Explanation: DROP COLUMN permanently removes one column from the table structure.

Solved Query 5

Question: Insert a new employee: EmpID 11, Name 'Rina', Dept 'HR', Salary 36000, City 'Cuttack', JoinDate '2023-01-15'.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
INSERT INTO Employee VALUES
(11, 'Rina', 'HR', 36000, 'Cuttack', '2023-01-15');
```

Expected Output:

1 row inserted – table now has 11 rows.

Explanation: All 6 values are given in the exact column order of the table.

Solved Query 6

Question: Insert an employee giving only EmpID, Name and Dept.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
INSERT INTO Employee (EmpID, Name, Dept)
VALUES (12, 'Farid', 'Finance');
```

Expected Output:

1 row inserted; Salary, City, JoinDate = NULL for this row.

Explanation: When a column list is given, only those columns receive values; the rest default to NULL.

Solved Query 7

Question: Insert three new employees in a single statement.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
INSERT INTO Employee VALUES
(13,'Neha','IT',47000,'Puri','2022-02-10'),
(14,'Tapan','Sales',31000,'Cuttack','2021-12-01'),
(15,'Lipsa','HR',39000,'Rourkela','2020-06-19');
```

Expected Output:

3 rows inserted in one INSERT statement.

Explanation: Multiple value sets, separated by commas, insert several rows in a single command.

Solved Query 8

Question: Increase Kiran's salary to 65000.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
UPDATE Employee
SET Salary = 65000
WHERE Name = 'Kiran';
```

Expected Output:

Kiran's Salary changes from 60000 to 65000; all other rows unaffected.

Explanation: WHERE Name = 'Kiran' ensures only that one row is updated.

Solved Query 9

Question: Update both City and Dept for employee with EmpID 6.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
UPDATE Employee
SET City = 'Bhubaneswar', Dept = 'HR'
WHERE EmpID = 6;
```

Expected Output:

Manoj's City becomes Bhubaneswar and Dept becomes HR.

Explanation: Multiple columns can be changed together in one SET clause, separated by commas.

Solved Query 10

Question: Give a 5% salary hike to all Sales department employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
UPDATE Employee
SET Salary = Salary + (Salary * 0.05)
WHERE Dept = 'Sales';
```

Expected Output:

Name	New Salary
Priya	31500
Manoj	34650
Ajay	30450

Explanation: SET Salary = Salary + (Salary*0.05) raises every matching row's salary by 5% using its own old value.

Solved Query 11

Question: Delete the employee record of Ajay.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
DELETE FROM Employee
WHERE Name = 'Ajay';
```

Expected Output:

Ajay's row is permanently removed; 9 rows remain.

Explanation: WHERE restricts DELETE to only the matching row.

Solved Query 12

Question: Delete all employees from the Rourkela city whose salary is below 35000.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
DELETE FROM Employee
WHERE City = 'Rourkela' AND Salary < 35000;
```

Expected Output:

Matching row(s), e.g. Ajay (29000, Rourkela), are removed.

Explanation: AND combines two conditions; both must be true for a row to be deleted.

Solved Query 13

Question: Display all details of every employee.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT * FROM Employee;
```

Expected Output:

All 10 rows, all 6 columns are displayed.

Explanation: The asterisk (*) means "every column".

Solved Query 14

Question: Display only Name and City of all employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, City FROM Employee;
```

Expected Output:

A two-column result with 10 rows (Name, City only).

Explanation: Listing specific column names returns only those columns.

Solved Query 15

Question: Display employees who work in the Finance department.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, Dept, Salary FROM Employee
WHERE Dept = 'Finance';
```

Expected Output:

Name	Dept	Salary
Kiran	Finance	60000
Pooja	Finance	55000

Explanation: WHERE Dept = 'Finance' filters only rows where the condition is true.

Solved Query 16

Question: Display employees who are in IT department AND earn more than 45000.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, Dept, Salary FROM Employee
WHERE Dept = 'IT' AND Salary > 45000;
```

Expected Output:

Name	Dept	Salary
Ramesh	IT	52000
Suresh	IT	48000

Explanation: AND requires BOTH conditions to be true for a row to appear in the result.

Solved Query 17

Question: Display employees whose salary is between 30000 and 45000.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, Salary FROM Employee
WHERE Salary BETWEEN 30000 AND 45000;
```

Expected Output:

Name	Salary
Anil	45000
Sunita	38000
Priya	30000
Manoj	33000
Deepa	41000

Explanation: BETWEEN...AND is inclusive of both boundary values (30000 and 45000 are included).

Solved Query 18

Question: Display employees who belong to IT or HR department.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, Dept FROM Employee
WHERE Dept IN ('IT','HR');
```

Expected Output:

Name	Dept
Anil	IT
Sunita	HR
Ramesh	IT
Deepa	HR
Suresh	IT

Explanation: IN checks membership in a list of values – shorter than writing multiple OR conditions.

Solved Query 19

Question: Display employees whose name starts with the letter 'S'.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name FROM Employee
WHERE Name LIKE 'S%';
```

Expected Output:

Name
Sunita
Suresh

Explanation: LIKE 'S%' matches any name that begins with S, regardless of what follows.

Solved Query 20

Question: Display the distinct list of cities where employees live.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT DISTINCT City FROM Employee;
```

Expected Output:

City
Bhubaneswar
Cuttack
Rourkela
Puri

Explanation: DISTINCT removes duplicate city names, showing each city only once.

Solved Query 21

Question: Find the total number of employees in the table.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT COUNT(*) FROM Employee;
```

Expected Output:

COUNT(*)
10

Explanation: COUNT(*) counts every row in the table, regardless of NULL values.

Solved Query 22

Question: Find the total salary paid to all employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT SUM(Salary) FROM Employee;
```

Expected Output:

SUM(Salary)
431000

Explanation: SUM() adds up every value in the Salary column.

Solved Query 23

Question: Find the average salary of all employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT AVG(Salary) FROM Employee;
```

Expected Output:

AVG(Salary)
43100

Explanation: AVG() = SUM(Salary) ÷ COUNT(Salary) = 431000 ÷ 10.

Solved Query 24

Question: Find the highest salary among all employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT MAX(Salary) FROM Employee;
```

Expected Output:

MAX(Salary)
60000

Explanation: MAX() scans the whole Salary column and returns the single largest value.

Solved Query 25

Question: Find the lowest salary among all employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT MIN(Salary) FROM Employee;
```

Expected Output:

MIN(Salary)
29000

Explanation: MIN() scans the whole Salary column and returns the single smallest value.

Solved Query 26

Question: Find the number of employees in each department.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept;
```

Expected Output:

Dept	COUNT(*)
IT	3
HR	2
Sales	3
Finance	2

Explanation: GROUP BY Dept creates one group per unique department; COUNT(*) counts rows in each group.

Solved Query 27

Question: Find the total salary paid in each department.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, SUM(Salary)
FROM Employee
GROUP BY Dept;
```

Expected Output:

Dept	SUM(Salary)
IT	145000
HR	79000
Sales	92000
Finance	115000

Explanation: SUM(Salary) is calculated separately within each Dept group.

Solved Query 28

Question: List each department's employee count, sorted from most to least employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept
ORDER BY COUNT(*) DESC;
```

Expected Output:

Dept	COUNT(*)
IT	3
Sales	3
HR	2
Finance	2

Explanation: GROUP BY forms the groups first; ORDER BY then sorts the grouped result by COUNT(*), highest first.

Solved Query 29

Question: Find departments that have more than 2 employees.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, COUNT(*)
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 2;
```

Expected Output:

Dept	COUNT(*)
IT	3
Sales	3

Explanation: HAVING filters the grouped results, keeping only departments where the count exceeds 2.

Solved Query 30

Question: Find departments with average salary above 40000, listed from highest to lowest average.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, AVG(Salary)
FROM Employee
GROUP BY Dept
HAVING AVG(Salary) > 40000
ORDER BY AVG(Salary) DESC;
```

Expected Output:

Dept	AVG(Salary)
Finance	57500.0
IT	48333.33

Explanation: This combines all four clauses: GROUP BY forms groups, HAVING filters groups by AVG, ORDER BY sorts the final result.

Solved Query 31

Question: List all employees sorted by salary, highest paid first.

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Name, Salary FROM Employee
ORDER BY Salary DESC;
```

Expected Output:

Name	Salary
Kiran	60000
Pooja	55000
Ramesh	52000
Suresh	48000
Anil	45000
Deepa	41000
Sunita	38000
Manoj	33000
Priya	30000
Ajay	29000

Explanation: ORDER BY Salary DESC arranges rows from the highest salary to the lowest.

Solved Query 32

Question: List all employees sorted by Department (A-Z), and within each department by Salary (highest first).

Table used: Employee (10 rows, as shown at the start of the book)

SQL Query:

```
SELECT Dept, Name, Salary FROM Employee
ORDER BY Dept ASC, Salary DESC;
```

Expected Output:

Dept	Name	Salary
Finance	Kiran	60000
Finance	Pooja	55000
HR	Deepa	41000
HR	Sunita	38000
IT	Ramesh	52000
IT	Suresh	48000
IT	Anil	45000
Sales	Manoj	33000
Sales	Priya	30000
Sales	Ajay	29000

Explanation: The first ORDER BY column (Dept) decides the main grouping order; the second column (Salary) breaks ties within each department.

Exam Point: This bank covers all 11 listed topics with 32 fully solved queries – revise each Question line first and try writing the query yourself before checking the answer.

11 Quick Revision & SQL Cheat Sheet

DDL – Structure Commands

```
CREATE DATABASE db_name;
DROP DATABASE db_name;

CREATE TABLE t (col type, ...);
ALTER TABLE t ADD col type;
ALTER TABLE t MODIFY col type;
ALTER TABLE t DROP COLUMN col;
DROP TABLE t;
```

DML – Data Commands

```
INSERT INTO t VALUES (...);
INSERT INTO t (c1,c2) VALUES (v1,v2);

UPDATE t SET c1=v1 WHERE cond;

DELETE FROM t WHERE cond;

SELECT c1,c2 FROM t WHERE cond;
```

Aggregate Functions

Function	Meaning
COUNT()	Number of rows
SUM()	Total of values
AVG()	Mean of values
MAX()	Largest value
MIN()	Smallest value

Clause Order (as written)

```
SELECT ...
FROM ...
WHERE ...
GROUP BY ...
HAVING ...
ORDER BY ...;
```

Clause Order (execution / logical order)

```
FROM -> WHERE -> GROUP BY -> HAVING -> SELECT ->
ORDER BY
```

WHERE vs HAVING (1 line)

WHERE filters ROWS before grouping. HAVING filters GROUPS after aggregation and can use COUNT/SUM/AVG/MAX/MIN directly.

Operators for WHERE

=, >, <, >=, <=, <> | AND, OR, NOT | BETWEEN...AND | IN(...) | LIKE 'pattern' (% = any chars, _ = one char)

One-Line Exam Facts

- CREATE, ALTER, DROP = DDL (structure); INSERT, UPDATE, DELETE, SELECT = DML (data).
- DROP TABLE removes structure + data; DELETE removes only data, structure stays.
- UPDATE/DELETE without WHERE affects ALL rows – always double-check WHERE before running.
- Aggregate functions ignore NULLs, except COUNT(*) which counts every row.
- GROUP BY is used to apply aggregate functions separately to each group of rows.
- HAVING is used only for conditions on aggregate functions; WHERE cannot contain aggregate functions.
- ORDER BY defaults to ASC; DESC must be written explicitly for descending order.
- DISTINCT removes duplicate rows/values from a SELECT result.
- Every non-aggregated column in SELECT must appear in GROUP BY.
- Logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

Final Tip: Practise writing each of the 32 solved queries from memory, then check against the Employee table data given at the start of this book. Consistency with ONE sample table is what makes SQL concepts stick for the exam.